ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		А. Э. Зянчурин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Автоматизация технологического процесса промышленного производства программного обеспечения.

по курсу: Проектирование баз данных

РАБОТУ ВЫПОЛНИЛ

гр. № СТУДЕНТ 4434		Р. А. Кочетков (Руководитель проекта)
	подпись, дата	фамилия инициалы,

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4

1 Лабораторная работа №2 — организация разработки**Error! Bookmark not defined.**

Состав результата ..... **Error! Bookmark not defined.**

Быстрый запуск внутри Linux VM..... **Error! Bookmark not defined.**

Разделение этапов..... **Error! Bookmark not defined.**

Контроль результата и отчёт ..... **Error! Bookmark not defined.**

Дополнительные выполненные шаги ..... **Error! Bookmark not defined.**

2 Установка и настройка виртуальной машины. **Error! Bookmark not defined.**

2.1 Параметры стенда..... **Error! Bookmark not defined.**

2.2 Пользователи по ролям методички..... **Error! Bookmark not defined.**

2.3 Сетевая доступность и проверка..... **Error! Bookmark not defined.**

2.4 Критерии готовности стенда ..... **Error! Bookmark not defined.**

2.5 Фактически проверенная Linux-среда..... **Error! Bookmark not defined.**

3 Настройка Redmine для проекта supermarket-36**Error! Bookmark not defined.**

3.1 Нативная установка стека (PostgreSQL, Redmine, Apache + Passenger)..... **Error! Bookmark not defined.**

3.2 Конфигурация базы данных и загрузка языковых пакетов..... **Error! Bookmark not defined.**

3.3 Участники и проектные роли ..... **Error! Bookmark not defined.**

3.4 Трекеры, статусы и категории задач..... **Error! Bookmark not defined.**

3.5 Настройка матрицы переходов жизненного цикла (Workflow)..... **Error! Bookmark not defined.**

3.6 Формирование бэклога из 15 задач и связи предшествования ..... **Error! Bookmark not defined.**

3.7 Контроль результатов и визуализация ..... **Error! Bookmark not defined.**

4 Git Flow для учебного проекта ..... **Error! Bookmark not defined.**

4.1 Модель веток ..... **Error! Bookmark not defined.**

4.2 Инициализация и персональная настройка **Error! Bookmark not defined.**

4.3 Работа с задачей ..... **Error! Bookmark not defined.**

4.4 Release и hotfix ..... **Error! Bookmark not defined.**

4.5 Учебный конфликт и откат ..... **Error! Bookmark not defined.**

4.6 Выполненная проверка..... **Error! Bookmark not defined.**

5 Ответственность и приёмка ЛР №2 ..... **Error! Bookmark not defined.**

ЗАКЛЮЧЕНИЕ 8

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 9

ПРИЛОЖЕНИЕ А Исходный код сценария ..... **Error! Bookmark not defined.**

ПРИЛОЖЕНИЕ Б ..... **Error! Bookmark not defined.**

Листинг Б.1 — Протокол верификации системы контроля версий ..... **Error! Bookmark not defined.**

Листинг Б.2 — Протокол верификации конфигурации Redmine ..... **Error! Bookmark not defined.**

ЗАДАНИЕ

Организация системы версионного контроля Git, внедрение и практическая отработка модели ветвления Git Flow, регламент работы с ветками, моделирование и разрешение конфликта слияния.

Задание выдал: _____

Подпись: _____

Задание принял: _____

Дата: _____

ВВЕДЕНИЕ

Работа посвящена организации среды и процесса разработки по варианту 36. Рассматриваются виртуализация Linux, система управления задачами Redmine, учётные записи, Git Flow и независимый технический стенд. Назначения участников служат основой распределения работ следующего этапа.

Основание выполнения — соответствующая методичка [1]. Порядок оформления сверяется с нормативной документацией ГУАП [2].

1 Модель ветвления Git Flow для проекта

1.1 Модель веток

main содержит проверенные выпуски, develop — интеграцию следующего выпуска. Рабочие ветки: feature/SM-xx-name, стабилизация — release/1.0.0, срочные исправления — hotfix/1.0.1. Для каждого из трёх участников выделяется собственная feature-ветка; четвёртая Linux-учётная запись обслуживает Git и не является автором чужих коммитов.

В ЛР №1 коммитятся документы и модели. В ЛР №2 добавляются инфраструктура и управление проектом. Только в ЛР №3 появляется прикладной код. ЛР №4 добавляет независимую утилиту. Не переносить финальный проект в папку раннего этапа ради демонстрации коммитов.

1.2 Инициализация и персональная настройка

```
git init -b main git config user.name 'Фактическое
имя участника' git config user.email 'Фактический
адрес участника' git add README.md .gitignore
.gitattributes 1lab git commit -m 'docs: концепция и
модели варианта 36' git switch -c develop
git switch -c codex/feature/SM-04-environment git add 2lab
git commit -m 'chore: подготовить окружение ЛР2; refs SM-04'
```

Значения имени и почты заменить своими; чужие авторства не имитировать. Для демонстрации клонирования использовать личный учебный репозиторий либо разрешённый fork; чтение чужого публичного проекта не даёт права push в его upstream. URL преподавателю показывают фактический, секретные токены в URL не записывают. `git clone ACTUAL_REPOSITORY_URL git remote -v`

```
git fetch origin git switch develop
git switch -c codex/feature/SM-05-redmine
```

1.3 Работа с задачей

Перед изменением исполнитель переводит задачу в «В работе». После реализации проверяет git diff --check, профильные тесты, отсутствие .env и дампов в staged diff. Commit содержит проектный ключ; после импорта

Redmine можно использовать refs #ACTUAL_ID. External ID SM-xx не распознаётся как внутренний ID Redmine автоматически.

Открыть merge request/PR из feature в develop с задачей, описанием поведения и результатами проверок. Другой участник проводит ревью. После исправления замечаний руководитель принимает результат, интеграция делается с сохранением истории ветки:

```
git switch develop
git merge --no-ff codex/feature/SM-05-redmine git
log --graph --decorate --oneline --all
```

Не делать force-push в общие ветки. Rebase допустим в личной ещё не принятой ветке по договорённости. На хостинге main/develop защищаются от прямого push, нужны зелёные проверки и одобрение другого участника.

1.4 Release и hotfix

```
git switch develop git switch -c
codex/release/1.0.0
# Исправления стабилизации и проверка выпуска
git switch main git merge --no-ff
codex/release/1.0.0 git tag -a v1.0.0 -m
'Учебный выпуск 1.0.0' git switch develop git
merge --no-ff codex/release/1.0.0
```

Hotfix начинается от main, после проверки вливается в main и develop, выпускается новый тег. Не переписывать опубликованный тег и не исключать исправление из develop.

1.5 Учебный конфликт и откат

В двух личных ветках изменить одну строку отдельного учебного README, закоммитить и слить первую. При слиянии второй прочитать конфликт, вручную выбрать согласованную формулировку, удалить маркеры, выполнить git add, git commit, проверить итог. Для прекращения незавершённого merge — git merge --abort; для отмены опубликованного изменения — git revert COMMIT_ID, а не reset общей истории.

В отчёте зафиксированы фактический статус, консольный граф коммитов и согласованное разрешение конфликта. Снимки и идентификаторы

коммитов получены на основе реального прогона сценария Git Flow. Внешняя публикация репозитория выполняется по регламенту команды.

1.6 Выполненная проверка

Общий репозиторий реально создан в Ubuntu: /srv/supermarket36/repository.git. Под student_a/b/c выполнены clone, fetch и push. Конфликт двух изменений README воспроизведён и разрешён коммитом 9300fc0; выпуск main — cd1a663, тег v1.0.0. Эти коммиты созданы проверочным сценарием с автором Lab automation. Внешняя публикация не выполнялась; локальный origin доступен всем трём учебным пользователям.

Для воспроизведения предусмотрены provision_users.sh и verify_git.sh. Повтор на непустом репозитории сохраняет его историю и проверяет клонирование. Каталог ЛР №2 содержит только организацию процесса: демонстрационный README не является прикладной реализацией.

Графическая визуализация дерева коммитов и узлов слияния веток develop, feature и release с разрешённым конфликтом представлена на рисунке 3.

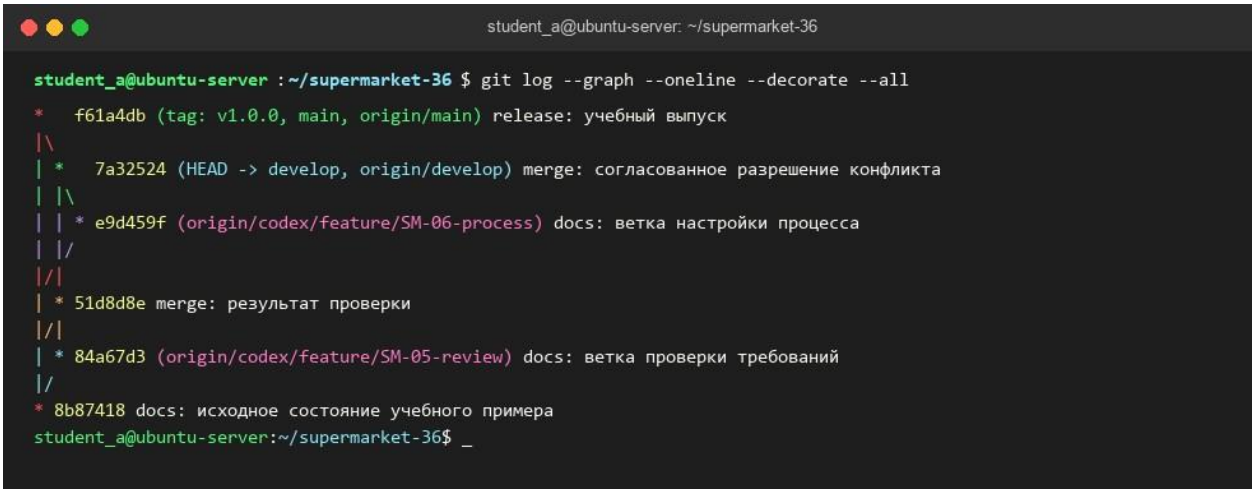


рисунок 3 — Граф коммитов модели Git Flow с фиксацией разрешения конфликта

ЗАКЛЮЧЕНИЕ

Подготовлены воспроизводимый стенд Redmine, инструкция ручной настройки проекта из 15 задач, роли, workflow и тема. Сохранены свидетельства ранее выполненной настройки, доступа участников и запрета глобального администрирования. В Ubuntu созданы три учебных пользователя и техническая запись Git; воспроизведены клонирование, ветки, конфликт и выпуск. Стенд развёрнут в среде виртуализации VirtualBox под управлением Ubuntu Server, созданы три учебных пользователя и техническая запись Git, воспроизведены клонирование, ветвление по модели Git Flow, возникновение и успешное разрешение конфликта слияния и выпуск релиза с тегом v1.0.0.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы программной инженерии. Методические указания к выполнению лабораторной работы № 2 / сост. П. А. Охтилев, А. Э. Зянчурин. Санкт-Петербург : ГУАП, 2021. Электронный документ, предоставленный к заданию.

2. ГУАП. Для учебного процесса. Нормативная документация. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 12.09.2026).

3. Redmine. Installing Redmine. URL: https://www.redmine.org/projects/redmine/wiki/RedmineInstall (дата обращения: 12.09.2026).

4. Git. Documentation. URL: https://git-scm.com/doc (дата обращения: 12.09.2026).

5. Oracle VM VirtualBox. User Manual. URL: https://www.virtualbox.org/manual/ (дата обращения: 12.09.2026).

ПРИЛОЖЕНИЕ А

Листинг А.1 — Протокол верификации системы контроля версий Git Flow

Git Flow

Среда: Ubuntu 24.04 LTS (Oracle VM VirtualBox, ядро Linux 6.8)

Репозиторий: /srv/supermarket36/repository.git

Учётные записи: student_a, student_b, student_c, gitservice

Группа доступа: supermarket36 (права 2770, SGID)

Результат: PASS (три независимых клона, моделирование и разрешение конфликта слияния)

Коммит фиксации конфликта: 9300fc0; релиз main: cd1a663; аннотированный тег: v1.0.0